C PROGRAMMING (CSA02)

ASSIGNMENT 1

Grama Jyothi Rural Electricity Co-operative

Name: S Dhana Sri

Register Number: 19252620062

Course: CSA0209

GitHub Repository Link: dhanasri-saravanan/1925260062-S-Dhana-sri-C-Programming

Mandatory Execution and GitHub Submission

- Every student must individually execute every task in this assignment, including all “Complete the Code” programs.
- Create a GitHub repository named CSA02-<RegisterNumber>-Assignment1 and share it with the course faculty. The repository may be public, or private with the faculty added as a collaborator.
- Keep one folder for each task (for example, Task01, Task02, … or Q1, Q2, …). Each folder must contain the C source code, input files and output screenshots.
- Include a README.md with your name, register number, compiler and version (for example, GCC 13), the exact GCC command used to compile, the steps to run each program, and a short note of the result of each task.
- Commit your work progressively, with at least one commit for each task. A single upload of all files on the last day will not be accepted. The commit history will be checked.
- Submit the GitHub link below and on the cover page of your report. An assignment submitted without a working GitHub link will not be evaluated.

How to Answer the Additional Questions

Complete the Code

Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output with the expected output.

Alternative Concept and Complexity Analysis

Describe how the solution would change if the alternative concept were used, with code or pseudocode. Analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Scenario: Grama Jyothi Rural Electricity Co-operative

Grama Jyothi is a village electricity co-operative that bills about 4,000 households every month. A meter reader records the previous and current readings of each house. The office computes the units consumed and the bill, which has a fixed charge and an energy charge that depends on slabs of consumption. For example, the first 100 units are free, the next 100 units cost Rs. 2.25 per unit, and every unit above 200 costs Rs. 4.50.

The present spreadsheet has produced wrong bills: some bills are off by a few paise, some households received a negative bill when their meter rolled over from 65535 to 0, and the tariff was typed in many places, so a tariff revision was applied only partly.

The number of readings processed may grow:

- 1 village: 4,000 houses
- 1 taluk: 60,000 houses
- 1 district: 800,000 houses

You are required to analyse the problem, choose suitable constants, data types and preprocessor features, implement the billing program in C and validate it, so that every bill is correct to the paisa.

Design Challenge

The co-operative is moving to smart meters that report every 15 minutes and to a time-of-day tariff. Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 1 – Implement and Test the Billing Program (4 Marks)

Bloom's Level: K3 – Apply

Implement the complete billing program and test it.

Required tests:

- Normal bills in each slab.
- Boundary cases: exactly 100 units, exactly 200 units and zero units.
- A meter that has rolled over from 65535 to 0.
- Invalid input: letters, a negative reading, and a current reading less than the previous one without roll-over.
- A small set of five bills whose total you calculate by hand.

Alternative Concept and Complexity Analysis

If the slab limits and rates were stored in two arrays and the bill computed by a loop over the slabs, instead of fixed if statements, describe the solution. Analyse the time per bill for s slabs (O(s) versus a fixed number of tests), the extra space, and explain which design is easier to change.

Evidence Expected

Working source code and a test table with expected and actual results for every case.

TASK 2 – Compare Expectations with Execution (4 Marks)

Bloom's Level: K4 – Analyse

Compare what you expected from your type and constant design with what actually happened. Discuss:

- Whether the float and the paise versions of the total for 4,000, 60,000 and 800,000 bills agree, and by how much they differ.
- Whether the int, long and long long totals overflow at district scale.
- How sizeof and the ranges on your system compare with the minimum sizes guaranteed by the C standard.
- Reasons for any difference.

Alternative Concept and Complexity Analysis

If the district total were accumulated in a double instead of a long long, explain what would happen at 800,000 bills. Analyse when each type loses accuracy (overflow versus precision loss), and compare the time per addition and the space per variable.

Evidence Expected

A reasoned comparison of expected and observed behaviour, supported by measured totals.

TASK 3 – Recommend and Defend the Final Design (4 Marks)

Bloom's Level: K5 – Evaluate

Recommend the final set of data types, constants and preprocessor features for the co-operative. Discuss:

- The types you recommend and why.
- How a tariff revision will be carried out.
- The limitations of your design.
- Whether the design remains correct at district scale.

Your recommendation must be based on evidence from your own experiments.

Alternative Concept and Complexity Analysis

If the tariff were read from a configuration file when the program starts, instead of being compiled into the program, explain the solution. Analyse the extra start-up time to read t tariff values and the space they occupy, against the cost of recompiling and redistributing the program after every revision.

Evidence Expected

A technically justified recommendation supported by your tests and measurements.

DAY 4 – DESIGN CHALLENGE: TASK 4 – Re-evaluate and Adapt the Design (8 Marks)

Bloom's Level: K5 – Evaluate

Changed Requirement

The co-operative is installing smart meters that send a reading in watt-hours every 15 minutes (35,040 readings per house per year). A time-of-day tariff applies: units used between 6 PM and 10 PM cost 20% more. The board also wants the tariff values to be changeable without recompiling the program.

Re-evaluate your original design. You may RETAIN, MODIFY or REDESIGN your original solution.

Suggested points to cover in your response

- State clearly whether you retain, modify or redesign the original solution and justify your decision.
- Explain how watt-hour readings will be stored and converted to billing units without losing precision.
- Describe how 15-minute time intervals will be classified as peak or off-peak.
- Explain how tariff values will be loaded from a configuration file and validated.
- Discuss data types, constants, overflow risks, accuracy, time complexity and space complexity.
- Estimate the annual reading volume for 4,000, 60,000 and 800,000 households.
- Describe test cases for time boundaries (6 PM and 10 PM), invalid configuration values, meter rollover and total-bill validation.

Submission Checklist

- ☐ Cover page includes student details and working GitHub repository link.
- ☐ Task folders contain source code, input files and output screenshots.
- ☐ All programs compile and execute successfully.

- ☐ Expected and actual results are recorded for every test.
- ☐ Complexity analyses include Big-O time and space.
- ☐ Measurements and conclusions are based on actual execution.
- ☐ README.md contains compiler details, compile commands, run instructions and task summaries.
- ☐ At least one meaningful commit exists for each task.

End of Assignment Brief